

P3071R1: Protection against modifications in contracts

Introduction

The expression that is checked for a true result in a contract annotation is supposed to observe the state of the program, but not change that state, exceptions such as logging notwithstanding. The current state of discussion of the contracts facility in SG21 offers little compile-time protections against accidental modifications. This paper proposes to add those by making *id-expressions* referring to local variables and parameters be const lvalues. Also, implicit or explicit references to `this` are as-if inside a const member function.

Example (only the lines marked "proposal" would change):

```
int global = 0;

int f(int x, int y, char *p, int& ref)
    pre((x = 0) == 0)           // proposal: ill-formed assignment to const lvalue
    pre((*p = 5))               // OK
    pre((ref = 5))              // proposal: ill-formed assignment to const lvalue
    pre(std::same_as_v<decltype(ref), int&>) // OK; yields true
    pre((global = 2))           // OK
    pre([x] { return x = 2; }()) // error: x is const
    pre([x] mutable { return x = 2; }()) // OK, modifies the copy of the parameter
    pre([&x] { return x = 2; }()) // proposal: ill-formed assignment to const lvalue
    pre([&x] mutable { return x = 2; }()) // proposal: ill-formed assignment to const lvalue
    post(r: y = r)              // error: y is not declared const
{
    contract_assert((x = 0));    // proposal: ill-formed assignment to const lvalue
    int var = 42;
    contract_assert((var = 42)); // proposal: ill-formed assignment to const lvalue

    static int svar = 1;
    contract_assert((svar = 1)); // OK
    return y;
}

struct S {
    int dm;
    void mf() /* not const */
        pre((dm = 1))           // proposal: ill-formed assignment to const lvalue
        pre([this]{ dm = 1; }()) // proposal: ill-formed assignment to const lvalue
        pre([this] () mutable { dm = 1; }()) // proposal: ill-formed assignment to const lvalue
        pre([*this]{ dm = 1; }()) // error: ill-formed assignment to const lvalue
        pre([*this] () mutable { dm = 1; }()) // OK, modifies the copy of *this
    {}
};
```

History

- R1: Added const treatment for lambda captures of `this`. R1 was approved by SG21 on 2023-12-14.

Proposal

Specifically, this paper proposes:

A *contract context* is the *conditional-expression* of a *contract-condition*, where the grammar non-terminals are as defined in P2961R2 (A natural syntax for Contracts).

1. An *id-expression* that is a subexpression of a contract context and names a variable with automatic storage duration of object type T , or a structured binding of type T whose corresponding variable has automatic storage duration, is an lvalue of type `const T`.
2. An *id-expression* that is a subexpression of a contract context and names a variable with automatic storage duration of type "reference to T " is an lvalue of type `const T`.
3. When a *lambda-expression* that is a subexpression of a contract context captures a non-function entity by copy, the type of the implicitly declared data member is T , but (as usual) naming such a data member in the *compound-statement* of the *lambda-expression* yields a `const` lvalue unless the lambda is declared *mutable*. When a *lambda-expression* captures such an entity by reference, the type of an expression naming the reference is `const T`.
4. The *primary-expression* `this`, when appearing as a subexpression of a contract context (including as the result of the implicit transformation in the body of a non-static member function), is a prvalue of type "pointer to cv X ", where cv is the combination of `const` and the *cv-qualifier-seq* of the enclosing member function (if any).
5. When a *lambda-expression* that is a subexpression of a contract context captures `this` of type "pointer to T ", the type of the implicitly declared data member is "pointer to `const T`".

Discussion

The following lists arguments in favor of the proposed change and those against, as a summary of earlier e-mail exchanges.

- Modifications inside contract conditions are discouraged. This change will enforce this rule at the outer level, because the type system of C++ prevents modifications through `const` lvalues.
- The `const` amendments are shallow (on the level of the lvalue only); attempting to invent "deep `const`" rules would make raw pointers and smart pointers likely behave differently, which is not desirable.
- The change enhances the bug resistance posture of C++. Typos such as `=` instead of `==` are caught more easily.
- If modifications are needed, `const_cast` can be applied, except that modifications of `const` objects continue to be undefined behavior (see [dcl.type.cv] p4). This includes parameters required to be declared `const` because they are used in a postcondition.
- Class members declared *mutable* can be modified as before; this is consistent with their behavior in `const` member functions.
- The type of lvalues referring to namespace-scope or local static variables is not changed; such accesses are more likely to be intentionally modifying, e.g. for logging or counting.
- The result of `decltype(x)` is not changed; aligned with `const` member functions, this yields the declared type of x . In contrast, `decltype((x))` yields `const T&`, where T is the type of the expression x .
- Expressions that are not lexically part of the contract condition are not changed. Overload resolution results (and thus, semantics) may change if code is hoisted from a contract condition into a separate function.
- Mechanically changing existing uses of `assert` to use `contract_assert` (e.g. by redefining a macro) might cause compilation failures because the argument of `assert` might not be `const`-correct. On the other hand, these compilation failures might indicate real bugs where program state is accidentally changed.
- Invoking a `const` member function may have different effects compared to invoking a non-`const` member function. Having source code with the same sequence of tokens in immediate proximity (inside vs. outside of `contract_assert`, for example) mean different things is confusing.

- If this paper is adopted, it is expected that an *id-expression* naming a return value identifier in a postcondition is also a const lvalue. This is not part of this proposal, though.
- A future extension might add contract captures. The behavior of such captures should be consistent with lambda captures in a contract context.